Trees & Isomorphism

Tree Definitions

Forest:

- An acyclic graph, built from a collection of separate unconnected trees

Tree:

- A connected acyclic graph (fully connected with no cycles)

Spanning Tree:

- A subgraph that is a tree that includes every vertex
- **Formal Definition:** A subgraph $G' = (V', E')$ of a connected graph $G = (V, E)$ such that $V' = V$ and G' is a tree

---

Tree Properties

Leaves:

- A vertex of degree 1
- Every tree of at least two vertices is guaranteed to have at least one leaf

Edge Count: ***(Important to Memorise)***

- A connected graph on n vertices is a tree *iff*, it only has exactly $n - 1$ edges

Unique Paths:

- For any two distinct vertices u and v in a tree T, there is a unique single path connecting them

---

Bipartite Nature

Every tree is inherently a bipartite graph

Method:

1. **Choose a Root:** Pick any vertex v and assign it to set V_1
2. **Measure Path Parity:** For every other vertex u, find the length of the unique path from v to u

3. **Distribute:** If the path length is odd, place u in set V_2, if the path length is even, place u in set V_1

Applications of Trees

- **Binary Search Trees:** Used for efficient sorting and data retrieval
- **Search Trees:** Used in AI to map out decision states (e.g. exploring future moves in Tic-Tac-Toe)
- **Phylogenetic Trees:** Used in Bioinformatics to map evolutionary relationships between species

---

Rooted Trees

A directed tree in which one vertex is fixed as the root, and every edge is directed away from this root

<u>Properties:</u>

- **Root:** The fixed vertex starting at level 0
- **Children:** The neighbours of a vertex v located in the next level down
- **Parent:** The unique neighbour of a vertex v in the previous level up

Internal Vertex:

- Any vertex that has children (including the root)

Level $L(i)$:

- The set of all vertices situated at exactly distance i from the root r

---

Rooted Tree Diagram:

```
[Level 0]                      (r) Root
                              /   \
                             /     \
                            v       v
[Level 1]                  (a)       (b) Internal Vertices
                          /   \         \
                         /     \         \
                        v       v         v
[Level 2]              (c)       (d)       (e) Internal Vertices
                      /           \         \
```

```
                    /             \         \
                   v               v         v
 [Level 3]        (f)              (g)       (h) <--- Leaves
```

Key Characteristics:

- **Directions:** Every edge points away from the root node (r)
- **Unique Paths:** There exists only 1 unique path from the root to any given node (because of the directions)
- **Levels:** Levels are determined from the distance (number of edges) from the root

---

Isomorphism

- Two graphs are isomorphic if they are structurally identical, regardless of how they are drawn or labelled

Formal Definition:

- Two graphs $G = (V, E)$ and $G' = (V', E')$ are isomorphic $(G \quad G')$ if there exists a bijective function $f : V \to V'$ such that for every $u, v \in V$, we have $uv \in E$ if and only if $f(u)f(v) \in E'$

"For every connected pair of vertices in G, when the bijective function maps those vertices to G' there is still an edge that connects the two vertices"

Necessary, but Not Sufficient Condition:

- Graphs that are isomorphic share all structural characteristics (number of edges, degree sequences, Eulerian Circuits etc)
- However, sharing these characteristics doesn't guarantee isomorphism (can only prove *non-isomorphism*)

Isomorphism on Rooted Trees

- The trees must have the same graph structure, ***AND*** their roots must be perfectly mapped to one another

Formal Definition:

- Two rooted trees $T_1(V_1, E_1, r_1)$ and $T_2(V_2, E_2, r_2)$ are isomorphic if there exists an isomorphism bijection $f : V_1 \quad V_2$ such that $f(r_1) = r_2$

Algorithm: Labelled Tree Isomorphism

This algorithm determines if two rooted trees are isomorphic by assigning string labels for every vertex from the bottom up
These string labels are combined into a single string of `1`s and `0`s and are used to check for isomorphism

Method:

1. **Base Case (Leaves):** Identify every leaf and assign each one the base string of `"10"`
2. **Internal Nodes:** Take an internal vertex and recursively retrieve the generated string labels from all of its immediate children
3. **Sort the Child Labels:** Sort the child strings into decreasing order (neutralises the differences in how the left-to-right branches are drawn)
4. **Label the Parent:** Concatenate the sorted child nodes together, prepend a `"1"` to the front, and append a `"0"` to the end (e.g. `["1100", "10"]` $\rightarrow$ `11100100`)
5. **Compare the Roots:** If the final string of trees T_1 and trees T_2 are the same, then the two trees are isomorphic

---

Algorithm in a Diagram

Label the Leaves:

```
          (r)
         /   \
       (a)   (e)
      /   \     \
    (b)   (c)   (f)
   "10"    |    / \
          (d) (g) (h)
         "10""10" "10"
```

<u>First internal Layer (Level 2):</u>

- The nodes `(c)` and `(f)` sort their children and wrap them in `1...0`
  - `(c)` wraps `["10"]` $\rightarrow$ `"1100"`
  - `(f)` wraps `["10", "10"]` $\rightarrow$ `"110100"`

```
          (r)
         /   \
       (a)   (e)
      /   \     \
    (b)   (c)   (f)
   "10" "1100" "110100"
```

Second internal Layer (Level 1):

- Nodes `(a)` and `(e)` process their children
- `(a)` retrieves `(b)` and `(c)`. Sorts `["1100", "10"]`. Wraps → `"11100100"`
- `(e)` retrieves `(f)`. Sorts `["110100"]`. Wraps → `"11101000"`

```
            (r)
           /   \
         (a)   (e)
"11100100"       "11101000"
```

The Root (Level 0):

- The root `(r)` processes its children to generate the final string for the tree
- `(r)` retrieves `(a)` and `(e)`. Sorts `["11101000", "11100100"]`. Wraps → `"111101000111001000"`

```
        (r)
"111101000111001000"
```

- *This final result can then be compared against the final strings of other root trees to determine isomorphism*

Algorithm Code Implementation:

```python
def label_vertex(T, v):
    # 1: if v is a leaf of T then
    if is_leaf(T, v):
        # 2: label(v) <- "10"
        label_v = "10"

    # 3: else
    else:
        l = []
        # 4: for every child w of v do
        for w in get_children(T, v):
            # 5: l(w) <- LABEL_VERTEX(T, w)
            l.append(label_vertex(T, w))

        # 6: Sort the labels of the children of v decreasingly
        l.sort(reverse=True)

        # 7: label(v) <- "1" l(w_1) l(w_2) ... l(w_k) "0"
        label_v = "1" + "".join(l) + "0"
```

```
    # 8: return label(v)
    return label_v

def labeled_tree_isomorphism(T1, r1, T2, r2):
    # 1: label(r1) <- LABEL_VERTEX(T1, r1)
    label_r1 = label_vertex(T1, r1)

    # 2: label(r2) <- LABEL_VERTEX(T2, r2)
    label_r2 = label_vertex(T2, r2)

    # 3: if label(r1) == label(r2) then
    if label_r1 == label_r2:
        # 4: return YES
        return "YES"

    # 5: else
    else:
        # 6: return NO
        return "NO"
```

- ***Note:** The comments are the Pseudocode from the lecture*

Related Pages:

- Intro to Graph Theory
- Shortest Paths, Cycles & Connectivity
- Breadth-First Search
- Depth-First Search